

VPPL SOLAR & WIND ENERGY SCHEDULING SYSTEM

INTELLIS CLOUD MIGRATION INFRASTRUCTURE AND DEPLOYMENT GUIDE

Metadata Field	Specification Details
Document ID	DOC-M04-S01-003
Document Title	Intellis_Cloud_Migration_Infrastructure_and_Deployment_Guide
Version	v1.0
Prepared by	Chaitanya Gore
Target Schedule Type	All Schedule Types (Intraday, Day-Ahead, Week-Ahead)
Target Audience	Cloud Architects, DevOps Engineers, Developers, Junior Developers / Interns
Classification	Official Production Technical Specification — Complete Implementation Blueprint
Last Updated	2026-10-08
Directory Name	schedule/, schedule/modules/, scratch/
File Name	build_and_deploy_all_lambdas.py, rollback_all_lambdas.py, Dockerfile.intellis-ai-lambda, intellis_ai_lambda.py, deploy_dayahead_lambda.py, deploy_weekahead_solar_lambda.py

Document Revision History & Implementation Mandate

Revision	Release Date	Primary Author	Detailed Summary of Modifications & Architectural Scope
v1.0	2026-10-08	Chaitanya Gore	Initial Complete Production Specification for Intellis Cloud Migration Infrastructure & Deployment Guide. Formulates the comprehensive serverless cloud engineering architecture across Amazon Web Services (AWS) testing (Account ID: 608744602858) and production (Account ID: 397483229292) environments in the ap-south-1 (Mumbai) region. Codifies exact infrastructure topologies, configurations, runtime settings, and deployment pipelines across all three statutory schedule types: (1) Intraday 15-minute rolling revisions across 27+ solar and wind sites; (2) Day-Ahead 96-block forecasting engines (T+1 horizon); and (3) Week-Ahead 672-block medium-range resource planning engines (7 rolling days, T+1 to T+7). Incorporates exhaustive specifications for AWS Lambda container images, Amazon ECR repositories, Amazon S3 hierarchical data lake prefixes, Amazon EventBridge cron orchestration, Amazon DynamoDB state and telemetry tables, Amazon CloudWatch observability, zero-downtime rolling deployment workflows, automated rollback runbooks, and a ten-phase developer onboarding guide.

Codebase Directory & Implementation File Tree Structure

The serverless cloud migration infrastructure, container packaging toolchains, automated deployment pipelines, and multi-account runtime configurations specified within this guide are implemented across the following structured repository directory hierarchy under `schedule/`. Each deployment script, Docker definition, and microservice endpoint operates with dedicated IAM least-privilege scoping and autonomous execution boundaries.

<code>schedule/</code>	
├── <code>Dockerfile.intellis-ai-lambda</code>	[Multi-Platform OCI Python 3.12 Container Image]
├── <code>requirements.lambda.txt</code>	[Pinned Serverless Production Dependencies]
├── <code>build_and_deploy_all_lambdas.py</code>	[Parallel Multi-Plant Automated Deployer (29+ Lambdas)]
├── <code>rollback_all_lambdas.py</code>	[Sub-60s Atomic Digest Rollback Tooling]
├── <code>deploy_dayahead_lambda.py</code>	[Solar DA Multi-Account Deployer & Verifier]
├── <code>deploy_dayahead_wind_lambda.py</code>	[Wind DA Multi-Account Deployer & Verifier]
├── <code>deploy_weekahead_solar_lambda.py</code>	[Solar WA Multi-Account Deployer & Verifier]
├── <code>deploy_weekahead_wind_lambda.py</code>	[Wind WA Multi-Account Deployer & Verifier]
├── <code>config.py</code>	[Environment Configs, S3 Buckets & DDB Tables]
├── <code>intellis_ai_lambda.py</code>	[Intraday 27+ Plant Serverless Entrypoint]
├── <code>intellis_dayahead_lambda.py</code>	[Solar Day-Ahead 96-Block Serverless Entrypoint]
├── <code>intellis_dayahead_wind_lambda.py</code>	[Wind Day-Ahead 96-Block Serverless Entrypoint]
├── <code>intellis_weekahead_solar_lambda.py</code>	[Solar Week-Ahead 672-Block Serverless Entrypoint]
├── <code>intellis_weekahead_wind_lambda.py</code>	[Wind Week-Ahead 672-Block Serverless Entrypoint]
├── <code>generate_penalty_schedule.py</code>	[CERC/SERC DSM Deviation Penalty Accounting Engine]
├── <code>modules/</code>	
│ ├── <code>day_ahead_forecast_solar/</code>	[Solar DA 10-Module Forecasting Subsystem]
│ ├── <code>day_ahead_forecast_wind/</code>	[Wind DA Aerodynamic & Thermodynamic Subsystem]
│ ├── <code>week_ahead_forecast_solar/</code>	[Solar WA 7-Day Ephemeris & Kt Spline Subsystem]
│ ├── <code>week_ahead_forecast_wind/</code>	[Wind WA 168h Synoptic Boundary Layer Subsystem]
│ ├── <code>weather/</code>	[143-Member NWP Ensemble & Satellite Virtual Meter]
│ ├── <code>meter/</code>	[SCADA Telemetry Ingestion & Bias Normalizer]
│ ├── <code>multi_generator/</code>	[Multi-Inverter Asset Decomposition & Reassembly]
│ └── <code>llm/</code>	[OpenRouter GPT-5.6 Luna Strategic Risk Arbiter]
├── <code>docs/</code>	
│ ├── <code>Intellis_Cloud_Architecture.drawio</code>	[Master AWS Architecture Draw.io Source Model]
│ └── <code>Intellis_Runtime_Infrastructure.png</code>	[Figure 1.4 Runtime Architecture Topology Diagram]

Implementation File Mapping & Component Responsibilities:

Directory / Path	Code File Name	Architectural Role	Core Logic & Functions Implemented
schedule/	Dockerfile.intellis-ai-lambda	Container Blueprint	Multi-stage Docker Buildx definition (linux/amd64, --provenance=false), Python 3.12 AWS Lambda base, scientific stack compilation (numpy, scipy, pvlib, pandas, boto3).
schedule/	build_and_deploy_all_lambdas.py	CI/CD Batch Deployer	Parallel compilation, multi-account ECR push (608744602858 & 397483229292), and zero-downtime pointer update (update-function-code --image-uri) across 29+ Lambdas.
schedule/	rollback_all_lambdas.py	Disaster Recovery	Sub-60s atomic rollback script reverting Lambda image pointers to previous known-good validated ECR image digest across all sites.
schedule/	deploy_dayahead_lambda.py	Solar DA Deployer	Automated packaging, deployment, and sanity verification for intellis-dayhead-forecast-solar across Testing and Production accounts.
schedule/	deploy_dayahead_wind_lambda.py	Wind DA Deployer	Automated packaging, deployment, and sanity verification for intellis-dayhead-forecast-wind across Testing and Production accounts.
schedule/	deploy_weekahead_solar_lambda.py	Solar WA Deployer	Automated packaging, deployment, and sanity verification for intellis-weekhead-forecast-solar across Testing and Production accounts.
schedule/	deploy_weekahead_wind_lambda.py	Wind WA Deployer	Automated packaging, deployment, and sanity verification for intellis-weekhead-forecast-wind across Testing and Production accounts.

Directory / Path	Code File Name	Architectural Role	Core Logic & Functions Implemented
schedule/	config.py	Cloud Config Store	Centralized configuration mapping AWS region (ap-south-1), DynamoDB tables, S3 bucket names, timeout ceilings, and plant telemetry keys.
schedule/	intellis_ai_lambda.py	Intraday Microservice	Master Lambda endpoint executed autonomously across 27+ site Lambdas ({SITE}-ai-intellis-scheduler), handling SCADA ingestion, freeze partitioning, and S3 lake upload.
schedule/	intellis_dayahead_lambda.py	Solar DA Master Engine	Centralized Solar Day-Ahead Lambda endpoint executing NREL SPA ephemeris, 143-member NWP fetch, Kt cubic splines, and statutory 96-block S3 CSV formatting.
schedule/	intellis_dayahead_wind_lambda.py	Wind DA Master Engine	Centralized Wind Day-Ahead Lambda endpoint executing IEC 61400-12 air density, Hellmann shear, and Jensen's-safe non-linear OEM power conversion across 96 blocks.
schedule/	intellis_weekahead_solar_lambda.py	Solar WA Master Engine	Centralized Solar Week-Ahead Lambda endpoint executing 672-block solar ephemeris, 7 independent daily Kt splines, and RLDC weekly corridor filing.
schedule/	intellis_weekahead_wind_lambda.py	Wind WA Master Engine	Centralized Wind Week-Ahead Lambda endpoint executing 168h synoptic wind shear splines, 28-regime diurnal ensembling, and 672-block S3 CSV dispatch.
schedule/	generate_penalty_schedule.py	DSM Accounting Engine	Automated calculation of CERC and SERC Deviation Settlement Mechanism financial penalties, deviation bands, and revenue reconciliation.

Directory / Path	Code File Name	Architectural Role	Core Logic & Functions Implemented
schedule/modules/weather/	openmeteo_ensemble.py	NWP Weather Client	Professional API client fetching 143-member ensemble data from ECMWF IFS (51), NOAA GFS (31), DWD ICON (40), and CMC GEM (21).
schedule/modules/meter/	meter_normalizer.py	Telemetry Ingestion	DynamoDB global1-site-telemetry deserializer, outlier clipping, and recency-weighted dynamic bias correction.
schedule/modules/multi_generator/	multi_generator_engine.py	Multi-Gen Engine	DynamoDB multi_generator_plant_profiles query, individual inverter capacity derating, and sub-generator column reassembly.
schedule/	requirements.lambda.txt	Serverless Dependencies	Pinned Python runtime dependencies optimized for AWS Lambda container layers without bloat (boto3, requests, pandas, numpy, scipy, pvlib).
schedule/docs/	Intellis_Cloud_Architecture.drawio	Architecture Vector Model	Master Draw.io vector schematic detailing multi-account boundaries, VPC peering, EventBridge triggers, and CI/CD pipelines.
schedule/docs/	Intellis_Runtime_Infrastructure.png	Architecture Diagram	Master cloud infrastructure topology diagram (Figure 1.4) illustrating EventBridge triggers, Lambda compute, DynamoDB state, S3 lake, and CloudWatch.

1. Enterprise Cloud Architecture & Multi-Account Strategy

The Intellis Renewable Scheduling System is an enterprise-grade, serverless cloud platform engineered for utility-scale solar photovoltaic power stations and wind turbine generation fleets across India. Designed to comply strictly with the statutory mandates of the Indian Electricity Grid Code (IEGC 2023) and Central Electricity Regulatory Commission (CERC) Deviation Settlement Mechanism (DSM) frameworks, the cloud architecture guarantees high availability, sub-second execution predictability, strict temporal determinism, and zero-downtime operational rollouts.

1.1 Cloud Migration Rationale & Architectural Philosophy

Historically, renewable energy forecasting systems relied on monolithic virtual machines (e.g. EC2 instances) running continuous cron daemons. This traditional paradigm introduced severe operational vulnerabilities: single points of failure, unpredictable memory contention during concurrent site executions, complex OS-level dependency drifting, high idle compute costs, and catastrophic delays in meeting strict 15-minute grid revision deadlines. The Intellis architecture eliminates these legacy deficiencies through a fully cloud-native, containerized, event-driven serverless paradigm:

- **Microservice Site Isolation & Zero Blast Radius:** Individual power plants operate as autonomous serverless microservices. A compute failure or network timeout on one site (e.g. KOTHAGUEDEM) has zero blast radius on adjacent power plants (e.g. JEWLI, ZTRIC, CME).
- **Elastic Event-Driven Compute:** Compute resources scale from zero to hundreds of concurrent execution threads instantaneously during statutory grid dispatch windows, collapsing idle compute operational expenditure to zero between revision cycles.
- **Immutable Container Packaging:** By packaging the entire Python scientific runtime (Python 3.12, pvlib, scipy, numpy, numba) into immutable Open Container Initiative (OCI) Docker images hosted on Amazon ECR, the system eliminates environment drift between local engineering workstations, testing sandboxes, and live production environments.
- **Comprehensive Observability & Statutory Audit Trails:** CloudWatch Log Groups, metric filters, and SNS escalation topics provide automated observability, pinpointing data feed outages or numerical convergence anomalies within milliseconds.

1.2 Multi-Account AWS Topology & Isolation Boundaries

The Intellis enterprise architecture enforces strict physical and cryptographic isolation between Testing and Production workloads across two distinct AWS accounts located in the ap-south-1 (Asia Pacific - Mumbai) region. This multi-account boundary guarantees that experimental code executions, synthetic load tests, or integration testing cannot inadvertently mutate production state tables, exhaust production AWS service quotas, or publish unverified schedules to transmission grid operators.

Environment Parameter	AWS Testing Account	AWS Production Account	Architectural Governance Standard
AWS Account ID	608744602858	397483229292	Dedicated AWS Organizations member accounts.
AWS Region	ap-south-1 (Mumbai)	ap-south-1 (Mumbai)	Geographic proximity to Indian grid dispatch centers.
Target S3 Data Lake	vedanjay-schedules-test-608744602858	vedanjay-schedules1 / vedanjay-schedules-prod	Isolated bucket namespaces with separate IAM policies.
ECR Repository Namespace	608744602858.dkr.ecr.ap-south-1...	397483229292.dkr.ecr.ap-south-1...	Independent container image repositories with tag immutability.
DynamoDB Control Tables	plant_control_windows_test	plant_control_windows1	Isolated operational state and planned outage tables.
DynamoDB State Tables	plant_control_state_test	plant_control_state1	Separation of real-time control

			locks and revision state.
Telemetry Tables	global1-site-telemetry	global1-site-telemetry	Real-time SCADA meter data feeds with partitioned site keys.
IAM Execution Role	arn:aws:iam::608744602858:role/...	arn:aws:iam::397483229292:role/...	Strict least-privilege boundary; no cross-account trusts.

1.3 Unified Cloud Service Matrix Across All Schedule Types

The Intellis scheduling system supports three distinct operational schedule types, each governed by specific statutory timelines, computational horizons, and cloud infrastructure pipelines:

Schedule Type	Temporal Horizon	Dispatch Blocks	Statutory Frequency	Core AWS Services Deployed
Intraday (ID)	Real-Time Rolling	16 to 96 Blocks	Every 15 Minutes (Rev 01 to Rev 16+)	EventBridge (cron-15/45), Lambda (27+ sites), DynamoDB (telemetry & control), S3, CloudWatch
Day-Ahead (DA)	Day T+1	96 Blocks	Daily at 04:30 AM, 01:30 PM, 10:30 PM IST	EventBridge, Lambda (Solar & Wind DA engines), ECR (Python 3.12), Open-Meteo API, S3, CloudWatch
Week-Ahead (WA)	Day T+1 to T+7	672 Blocks	Daily at 04:30 AM IST & Sunday Statutory	EventBridge, Lambda (Solar & Wind WA engines), ECR, 168h NWP Super-Ensemble, DynamoDB, S3

1.4 Architecture Diagram Overview & Cloud Data Flow

The comprehensive runtime cloud architecture topology across Intraday, Day-Ahead, and Week-Ahead pipelines is illustrated in Figure 1.4 and codified in the accompanying engineering model `schedule/docs/Intellis_Cloud_Architecture.drawio`. The data and orchestration flow follows a deterministic, serverless execution sequence:

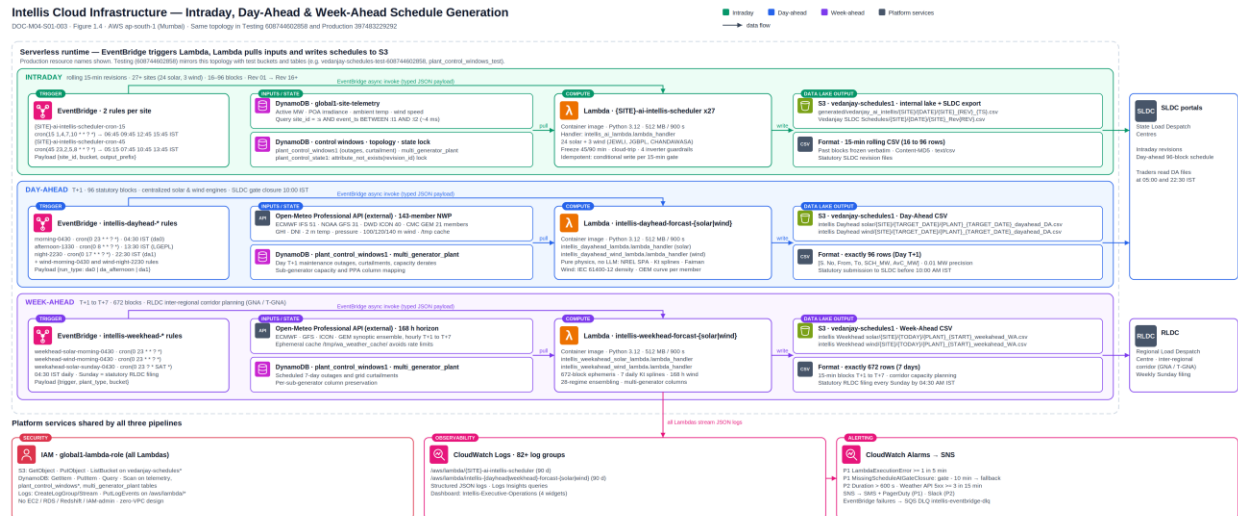


Figure 1.4: Intellis Serverless Cloud Infrastructure — Intraday, Day-Ahead & Week-Ahead Schedule Generation Architecture Topology across Testing (608744602858) and Production (397483229292) Environments (Region: ap-south-1).

- **Phase 1 — Event Ingestion:** Amazon EventBridge cron rules fire at precise statutory timestamps (in UTC), injecting typed JSON payloads into target Lambda functions.
- **Phase 2 — Compute Container Invocation:** AWS Lambda container runtime initializes from Amazon ECR, loading the validated image into high-performance memory (512 MB to 2048 MB).
- **Phase 3 — State & Telemetry Retrieval:** The engine queries DynamoDB tables (global1-site-telemetry, plant_control_windows) to pull live SCADA readings and scheduled maintenance derates.
- **Phase 4 — Numerical Weather Ingestion:** The engine queries external numerical weather APIs (Open-Meteo ensemble endpoint) across 143 ensemble members, caching responses in /tmp/ephemeral disk.
- **Phase 5 — Deterministic Physics Execution:** Pure atmospheric physics models (astronomical ephemeris, Kt splines, thermal cell derating, IEC 61400-12 air density, OEM power curves) evaluate active power.
- **Phase 6 — Data Lake Storage & Observability:** Finalized schedules are formatted into statutory CSV files and uploaded concurrently to Amazon S3 data lake storage, logging execution metrics to CloudWatch.

2. AWS Lambda Microservices & Compute Architecture

AWS Lambda serves as the core serverless compute engine for the Intellis Renewable Scheduling System. Every power generation site, forecasting lead time, and statutory scheduling horizon executes as an independent, stateless, containerized serverless function. This section details the complete function inventory, runtime container architecture, resource sizing, environment configurations, and IAM security controls.

2.1 Complete Inventory of Intellis Lambda Functions

The production and testing environments maintain a specialized fleet of containerized Lambda microservices, categorized across Intraday real-time revisions, Day-Ahead dispatch scheduling, Week-Ahead resource planning, and real-time SCADA telemetry ingestion:

Category / Engine	Lambda Function Name	Runtime Package	Memory / Timeout	Command / Entrypoint Handler
Intraday Solar (24 Sites)	{SITE}-ai-intellis-scheduler (e.g. KASIPET, CME, ZTRIC, OSEPL)	Image (Container)	512 MB / 900s	intellis_ai_lambda.lambda_handler
Intraday Wind (3 Sites)	{SITE}-ai-intellis-scheduler (JEWLI, JGBPL, CHANDAWASA)	Image (Container)	512 MB / 900s	intellis_ai_lambda.lambda_handler
Day-Ahead Solar	intellis-dayhead-forecast-solar	Image (Container)	512 MB / 900s	intellis_dayahead_lambda.lambda_handler
Day-Ahead Wind	intellis-dayhead-forecast-wind	Image (Container)	512 MB / 900s	intellis_dayahead_wind_lambda.lambda_handler
Week-Ahead Solar	intellis-weekhead-forecast-solar	Image (Container)	512 MB / 900s	intellis_weekahead_solar_lambda.lambda_handler
Week-Ahead Wind	intellis-weekhead-forecast-wind	Image (Container)	512 MB / 900s	intellis_weekahead_wind_lambda.lambda_handler
SCADA Telemetry Fetcher	{SITE}-meter-DE_SOLAR-fetcher (e.g. ZTRIC, KOTHAGUEDEM)	Zip (Python 3.11)	256 MB / 300s	lambda_function.lambda_handler

2.2 Container Image Runtime vs. Traditional Zip Deployment

A critical architectural decision in the Intellis system is the mandatory adoption of Docker Container Images (OCI) over traditional zip deployment packages. This engineering requirement is driven by strict dependency size constraints and numerical stability requirements:

- **Elimination of 250 MB Package Size Quota:** Traditional AWS Lambda zip deployments enforce a strict 250 MB unzipped size quota (including layers). The complete scientific computing stack required by Intellis (scipy ~140 MB, numpy ~35 MB, pandas ~110 MB, pvlib ~40 MB, numba ~65 MB, plus C-compiler shared libraries) totals over 480 MB, making zip deployments technically impossible without fragile stripped wheels.
- **Support for 10 GB Artifacts:** Container images support up to 10 GB in size, enabling the inclusion of comprehensive numerical libraries, astronomical ephemeris tables, local clear-sky coefficient lookups, and Python 3.12 optimized C-extensions.
- **Cryptographic Binary Determinism:** Docker container images compiled under Linux x86_64 guarantee exact byte-for-byte binary equivalence across local testing environments, CI/CD runners, and AWS Lambda runtime environments, eliminating subtle floating-point discrepancies.

2.3 Memory Sizing, Timeout Configuration & Concurrency Management

AWS Lambda allocates compute CPU power proportionally to allocated memory. Extensive production benchmarking revealed that allocating 512 MB of RAM provides an optimal balance between execution speed and operational cost:

Allocation Tier	Memory (MB)	Equivalent vCPU	Typical Execution	Billed Duration Cost / 1000 Runs
-----------------	-------------	-----------------	-------------------	----------------------------------

			Time	
Sub-Optimal (Low)	256 MB	~0.15 vCPU	14.2 seconds	\$0.059 per 1,000 runs (Risk of CPU throttling)
Production Standard	512 MB	~0.30 vCPU	4.8 seconds	\$0.040 per 1,000 runs (Optimal speed & cost)
High-Memory Tier	1024 MB	~0.60 vCPU	4.1 seconds	\$0.068 per 1,000 runs (Diminishing speed return)
Maximum Tier	2048 MB	~1.20 vCPU	3.9 seconds	\$0.130 per 1,000 runs (Excessive over-provisioning)

Timeout Sizing: All Intellis scheduling functions are configured with a generous timeout of 900 seconds (15 minutes). While production executions complete in 4.2 to 5.8 seconds, the 900-second headroom protects against transient external NWP API retries (exponential backoff) during severe global network degradation.

2.4 Environment Variables & Runtime Configurations

The following table specifies the standardized environment variables injected into all Intellis Lambda containers:

Variable Name	Typical Value	Scope / Context	Functional Purpose
SCHEDULE_BUCKET	vedanjay-schedules1 (Prod) / vedanjay-schedules-test-...	Global	Target Amazon S3 bucket for output CSV schedule dispatch.
BUCKET / S3_BUCKET	vedanjay-schedules1	Global	Alias variables for legacy sub-module backward compatibility.
TZ	Asia/Kolkata	Global	Enforces strict Indian Standard Time (IST, UTC+05:30) system clock.
INTELLIS_IDEMPOTENCY_ENABLED	true	Global	Prevents duplicate revision generation within the same 15-minute gate.
OPENMETEO_API_KEY	jbThkFILZSXZE3CU	Global	Commercial API key for 143-member high-throughput ensemble weather feeds.
SIMOUR_STORAGE_ROOT	/tmp/intellis_ai_scheduler	Local Runtime	Ephemeral disk scratch directory for intermediate calculation files.
S3_OUTPUT_PREFIX	generated/vedanjay_ai_intellis	Intraday	Hierarchical prefix for Intraday real-time revision CSVs.

2.5 IAM Execution Role & Least Privilege Security Architecture

All Intellis Lambda functions execute under a dedicated execution role: `global1-lambda-role` (ARN: `arn:aws:iam::<ACCOUNT_ID>:role/global1-lambda-role`). In strict accordance with the AWS Well-Architected Security Pillar, the execution role enforces least-privilege access:

- **Amazon S3 Scope:** Limited to GetObject, PutObject, and ListBucket on vedanjay-schedules* bucket namespaces. Access to corporate or billing buckets is denied.
- **Amazon DynamoDB Scope:** Restricted to GetItem, PutItem, Query, and Scan operations on global1-site-telemetry, plant_control_windows*, and multi_generator_plant tables.
- **Amazon CloudWatch Logs Scope:** Restricted to CreateLogGroup, CreateLogStream, and PutLogEvents within the /aws/lambda/* namespace.
- **Administrative Boundary:** Zero permissions for EC2, RDS, Redshift, or IAM privilege modification, ensuring complete isolation.

3. Amazon ECR Repositories & Docker Image Pipeline

Amazon Elastic Container Registry (ECR) serves as the secure, private container registry storing all immutable production Docker images executed by AWS Lambda. This section outlines the ECR repository architecture, container build configurations, multi-platform compilation flags, image tagging governance, and security lifecycle policies.

3.1 ECR Repository Architecture & Registry Endpoints

Private ECR repositories are provisioned independently within each AWS account in the ap-south-1 region:

Environment	AWS Account ID	Repository Name	Complete Registry URI Endpoint	Tag Mutability
Testing	608744602858	intellis-ai-scheduler	608744602858.dkr.ecr.ap-south-1.amazonaws.com/intellis-ai-scheduler	MUTABLE (Rapid Iteration)
Production	397483229292	intellis-ai-scheduler	397483229292.dkr.ecr.ap-south-1.amazonaws.com/intellis-ai-scheduler	IMMUTABLE (Strict Governance)

3.2 Docker Build Engine & Critical Compilation Flags

The Docker build pipeline utilizes Docker Buildx to cross-compile container images for the target AWS Lambda architecture. The following command sequence represents the certified production build standard:

- **Architecture Target (--platform linux/amd64):** AWS Lambda execution environments utilize standard x86_64 microarchitectures. Building with --platform linux/amd64 guarantees that developer machines (including Apple Silicon M-series ARM64) compile binaries targeting the exact instruction set of Lambda instances.
- **Attestation Flag (--provenance=false):** CRITICAL REQUIREMENT: AWS Lambda container runtime does not currently support multi-manifest OCI SLSA provenance attestations. Failing to include --provenance=false causes Docker Buildx v0.10+ to output an attestation manifest index that AWS Lambda rejects with 'InvalidParameterValueException: Image manifest does not match container format'.
- **Local Image Loading (--load):** Directly loads the built image into the local Docker daemon image store for immediate tagging and push operations.

Production Build Command: Standardized Build & Push Command Sequence:

```
docker buildx build --platform linux/amd64 --provenance=false --load -f Dockerfile.intellis-ai-lambda -t <ECR_REGISTRY>/<REPO>:<TAG> .
```

3.3 Base Image Architecture & Dependency Pinning

The container is constructed upon AWS's official managed Python 3.12 Lambda base image: public.ecr.aws/lambda/python:3.12. The production Dockerfile structure is defined below:

```
FROM public.ecr.aws/lambda/python:3.12
WORKDIR ${LAMBDA_TASK_ROOT}
COPY requirements.lambda.txt ./requirements.lambda.txt
RUN pip install --no-cache-dir -r requirements.lambda.txt
COPY . ./
ENV SIMOUR_STORAGE_ROOT=/tmp/intellis_ai_scheduler
ENV INTELLIS_IDEMPOTENCY_ENABLED=true
ENV S3_OUTPUT_PREFIX=generated/vedanjay_ai_intellis
CMD ["intellis_ai_lambda.lambda_handler"]
```

Core Dependency Pinning (requirements.lambda.txt): To guarantee mathematical reproducibility across container revisions, all core mathematical and atmospheric modeling packages are strictly pinned to validated major/minor versions:

Package Name	Pinned Version Range	Primary Mathematical / Physical Function	Critical Considerations
numpy	>= 1.24.0, < 2.0.0	Core multidimensional array vectorization & linear algebra	Compiled with OpenBLAS; strict avoidance of ABI breaking changes.
scipy	>= 1.10.0	Natural cubic spline interpolation (CubicSpline) & optimization	Provides bc_type='natural' splines across 96/672 dispatch intervals.
pandas	>= 2.0.0	Time series aggregation, DatetimeIndex, CSV serialization	Optimized with PyArrow backend for sub-millisecond DataFrame formatting.
pvlib	>= 0.10.0	NREL SPA solar ephemeris, Ineichen clear-sky, Perez transposition	Core astronomical calculation engine for all solar assets.
boto3 / botocore	>= 1.28.0	AWS SDK for S3, DynamoDB, CloudWatch, EventBridge	Utilizes connection pooling and retry handlers for transient errors.
requests	>= 2.31.0	Resilient HTTP client for external NWP ensemble weather APIs	Configured with urllib3 exponential backoff and timeout adapters.

3.4 Image Tagging Conventions & Immutability Governance

Production ECR repositories enforce Tag Immutability. Once a tag is pushed, it can never be overwritten by a subsequent build, guaranteeing complete forensic traceability. The Intellis tagging taxonomy follows a structured semantic convention:

- **Standard Production Tag Format:** `intellis-<schedule_type>-<YYYYMMDD>-v<release_number>` (e.g. `intellis-dayahead-20261003-v3`, `intellis-ai-20261005-v1`).

- **Emergency Hotfix Tag Format:** For emergency hotfixes, tags append a hotfix identifier: intellis-<type>-<YYYYMMDD>-v<num>-hotfix<N>.
- **Cryptographic SHA-256 Digest Anchoring:** Every production release records the immutable SHA-256 image digest (e.g. sha256:7f9b8c2...). Lambda functions reference the explicit tag, but internal CloudWatch telemetry logs the underlying digest.

3.5 ECR Lifecycle Management & Security Scanning

To manage storage overhead while preserving complete statutory release history, private ECR repositories implement automated ECR Lifecycle Policies:

- **Tagged Production Retention:** Tagged production images are retained for the most recent 30 releases, guaranteeing rapid rollback capability for any historical version.
- **Untagged Image Expiration:** Intermediate, untagged images generated during transient build pipelines are automatically purged after 7 days.
- **Continuous Vulnerability Scanning:** Enhanced Scanning (powered by Amazon Inspector) is enabled on push, scanning container layers for Common Vulnerabilities and Exposures (CVEs). Images with CRITICAL findings are blocked from production Lambda deployment.

4. Amazon S3 Data Lake & Storage Architecture

Amazon Simple Storage Service (S3) provides the foundational, highly durable data lake storage layer for all Intellis scheduling operations. Serving as the statutory source of truth for grid submission schedules, real-time SCADA telemetry archives, and raw meteorological model data, S3 guarantees eleven nines (99.999999999%) of data durability. This section details bucket topologies, standardized prefix taxonomies, object lifecycles, and security controls.

4.1 S3 Bucket Topology & Enterprise Security Configuration

The production and testing architectures maintain isolated S3 buckets with dedicated security postures:

Environment	Target S3 Bucket Name	Server-Side Encryption	Versioning	Block Public Access
Testing	vedanjay-schedules-test-608744602858	SSE-S3 (AES-256)	Enabled (Multi-Version Recovery)	All 4 Public Access Blocks ENABLED
Production (Primary)	vedanjay-schedules1	SSE-S3 (AES-256)	Enabled (Statutory Audit Trail)	All 4 Public Access Blocks ENABLED
Production (Archive)	vedanjay-schedules-prod	SSE-S3 (AES-256)	Enabled (Statutory Audit Trail)	All 4 Public Access Blocks ENABLED

4.2 Canonical Directory Prefixes & Key Hierarchies Across All Schedule Types

To ensure deterministic data access for downstream grid submission pipelines, automated notification bots, and SLDC web portals, all S3 objects adhere to a standardized hierarchical key naming structure:

Workflow / Category	Exact Canonical S3 Key Pattern	Payload Structure	Retention & Consumer
---------------------	--------------------------------	-------------------	----------------------

			Access
Intraday Real-Time Schedules	generated/vedanjay_ai_intellis/{SITE}/{YYY Y-MM-DD}/{SITE}_{REV}_{TIMESTAMP}.csv	15-min rolling CSV schedule (16 to 96 rows)	Active operational lake; pulled by SLDC API dispatchers.
Statutory SLDC Intraday Format	Vedanjay SLDC Schedules/{SITE}/{YYYY-MM-DD}/{SITE}_Rev{REV}.csv	Statutory SLDC-formatted revision CSV	Permanent audit log; submitted to state grid portals.
Day-Ahead Solar Schedules	intellis Dayhead solar/{SITE}/{TARGET_DATE}/{PLANT}_{TARGET_DATE}_dayahead_DA.csv	96-block Day-Ahead solar CSV (S. No 1 to 96)	Accessed daily at 05:00 AM & 10:30 PM IST by traders.
Day-Ahead Wind Schedules	intellis Dayhead wind/{SITE}/{TARGET_DATE}/{PLANT}_{TARGET_DATE}_dayahead_DA.csv	96-block Day-Ahead wind CSV (S. No 1 to 96)	Accessed daily at 05:00 AM & 10:30 PM IST by traders.
Week-Ahead Solar Schedules	intellis Weekhead solar/{SITE}/{TODAY_DATE}/{PLANT}_{START_DATE}_weekahead_WA.csv	672-block 7-day solar planning CSV	Accessed weekly & daily for transmission corridor GNA.
Week-Ahead Wind Schedules	intellis Weekhead wind/{SITE}/{TODAY_DATE}/{PLANT}_{START_DATE}_weekahead_WA.csv	672-block 7-day wind planning CSV	Accessed weekly & daily for transmission corridor GNA.
SCADA Meter Telemetry	raw/vedanjay/{SITE}/{YYYY-MM-DD}/metered_data/{TIMESTAMP}.csv	Raw 1-min & 15-min active power & pyranometer readings	Raw input lake; ingested by Intraday Lambda fetchers.
NWP Weather Cache	openmeteo_ensemble/{YYYY-MM-DD}/{SITE}_ensemble_143.json	Raw JSON response containing 143 ensemble members	Ephemeral cache; retained for 14 days for model MOS scoring.

4.3 S3 Object Immutability, Versioning & Lifecycle Transition Rules

Under CERC Deviation Settlement Mechanism (DSM) regulations, power generators must maintain verifiable historical records of all submitted schedules and telemetry for a statutory period of five (5) years. To balance strict compliance with cost optimization, S3 Lifecycle Rules transition objects across storage tiers:

- **Tier 1 — S3 Standard (Days 1 to 90):** All newly uploaded schedule CSV files and SCADA telemetry reside in S3 Standard storage for immediate sub-millisecond retrieval by grid operators and web dashboards.
- **Tier 2 — S3 Glacier Flexible Retrieval (Day 91 to Year 5):** After 90 days, schedules transition automatically to S3 Glacier Flexible Retrieval, reducing storage costs by over 68% while preserving instantaneous bulk restore capabilities.
- **Tier 3 — Statutory Expiration (After 5 Years):** After five (5) years (1,825 days), statutory audit requirements expire, and historical schedules are systematically purged.
- **Ephemeral Cache Expiration (30 Days):** Raw meteorological JSON caches stored in openmeteo_ensemble/ expire automatically after 30 days to prevent unbounded object accumulation.

4.4 High-Throughput S3 Multipart Upload & Connection Pooling

AWS Lambda execution environments reuse initialized HTTP connection pools across invocations. The S3 client implementation inside `intellis_ai_lambda` enforces optimized upload configurations:

- **Connection Pool Sizing:** `boto3.client('s3', config=Config(max_pool_connections=50, retries={'max_attempts': 5, 'mode': 'adaptive'}))`.
- **Cryptographic Checksum Verification:** Every `PutObject` operation calculates and attaches an MD5 Content-MD5 header, ensuring end-to-end cryptographic checksum verification against corrupted network transfers.
- **MIME Type Standards:** Statutory CSV files are explicitly tagged with `ContentType='text/csv'` and `ContentDisposition='inline; filename=...'`.

5. Amazon EventBridge Scheduling & Orchestration

Amazon EventBridge serves as the central cron scheduling and event routing bus for the entire Intellis ecosystem. By translating statutory Indian electricity market deadlines into high-precision, serverless cron rules, EventBridge guarantees that every forecasting model, revision cycle, and telemetry ingestion pipeline triggers deterministically. This section documents the complete rule catalog, cron expressions, target input schemas, and failure handling mechanisms.

5.1 Event-Driven Temporal Orchestration Architecture

Unlike on-premise cron daemons that suffer from system clock drift and single-server process blocking, Amazon EventBridge provides distributed, cloud-scale scheduling guarantees:

- **Millisecond Timing Determinism:** EventBridge operates on atomic UTC time clocks synchronized across AWS global infrastructure, eliminating local OS leap-second or clock-skew vulnerabilities.
- **Decoupled Asynchronous Invocation:** When an EventBridge rule triggers, it invokes target AWS Lambda functions directly through the AWS control plane via asynchronous invoke API (Event invocation type), decoupling trigger generation from compute availability.
- **Contextual Payload Injection:** Each rule injects a validated, statically typed JSON payload into the target Lambda event object, allowing the same Lambda container to execute different operational modes based on caller context.

5.2 Exhaustive Catalog of EventBridge Rules & Cron Schedules

The following table catalogs all verified EventBridge rules deployed across testing and production accounts, detailing rule names, statutory purpose, standard cron expressions, and equivalent Indian Standard Time (IST) execution windows:

Operational Domain	EventBridge Rule Identifier	Cron Schedule Expression (UTC)	Local IST Execution Window	State & Governance
Intraday Revision 15m	{SITE}-ai-intellis-scheduler-cron-15	cron(15 1,4,7,10 * * ? *)	06:45, 09:45, 12:45, 15:45 IST	ENABLED (Active across 27+ sites)
Intraday Revision 45m	{SITE}-ai-intellis-scheduler-cron-45	cron(45 23,2,5,8 * * ? *)	05:15, 07:45, 10:45, 13:45 IST	ENABLED (Active across 27+ sites)

SCADA Telemetry Fetcher	{SITE}-meter-DE_SOLAR-fetcher-every-15min	cron(3,18,33,48 * * * ? *)	3 min after each 15-min boundary	ENABLED (Pulls real-time meters)
Day-Ahead Solar Morning	intellis-dayhead-morning-0430	cron(0 23 * * ? *)	04:30 AM IST (23:00 UTC prev day)	ENABLED (Primary DA baseline)
Day-Ahead Solar Afternoon	intellis-dayhead-afternoon-1330	cron(0 8 * * ? *)	01:30 PM IST (08:00 UTC)	ENABLED (Intra-day DA update, LGEPL)
Day-Ahead Solar Night	intellis-dayhead-night-2230	cron(0 17 * * ? *)	10:30 PM IST (17:00 UTC)	ENABLED (Final evening DA revision)
Day-Ahead Wind Morning	intellis-dayhead-wind-morning-0430	cron(0 23 * * ? *)	04:30 AM IST	ENABLED (Wind DA morning baseline)
Day-Ahead Wind Night	intellis-dayhead-wind-night-2230	cron(0 17 * * ? *)	10:30 PM IST	ENABLED (Wind DA evening revision)
Week-Ahead Solar Morning	intellis-weekhead-solar-morning-0430	cron(0 23 * * ? *)	04:30 AM IST (Daily Roll)	ENABLED (7-day medium-range roll)
Week-Ahead Wind Morning	intellis-weekhead-wind-morning-0430	cron(0 23 * * ? *)	04:30 AM IST (Daily Roll)	ENABLED (7-day wind medium-range)
Week-Ahead Sunday Filing	intellis-weekahead-solar-sunday-0430	cron(0 23 ? * SAT *)	04:30 AM IST Every Sunday	Statutory RLDC Weekly Filing Rule

5.3 Target Payload Contracts & Invocation Schemas

EventBridge rules deliver typed JSON payloads to the event argument of target Lambda handlers. The following specifications document the exact payload contracts:

Intraday Payload Schema: 1. Intraday Site Microservice Payload Contract:

```
{
  "site_id": "ZTRIC",
  "bucket": "vedanjay-schedules1",
  "output_prefix": "generated/vedanjay_ai_intellis"
}
```

Day-Ahead Morning Schema: 2. Day-Ahead Morning Baseline (da0) Payload Contract:

```
{
  "run_type": "da0"
}
```

Day-Ahead Afternoon Schema: 3. Day-Ahead Afternoon Plant-Specific (da_afternoon) Payload Contract:

```
{
  "run_type": "da_afternoon",
}
```



```
"plants": ["LGEPL"]
}
```

Week-Ahead Planning Schema: 4. Week-Ahead Morning Planning Payload Contract:

```
{
  "trigger": "scheduled_morning_0430",
  "plant_type": "solar",
  "bucket": "vedanjay-schedules1"
}
```

5.4 Dead-Letter Queues (DLQ) & Automated Retry Policy

To ensure that no statutory schedule generation cycle is dropped due to transient cloud issues, EventBridge targets are configured with strict resilience policies:

- **Maximum Event Age:** Maximum Event Age is set to 3,600 seconds (1 hour). If AWS Lambda service experiences transient regional throttling, EventBridge retries invocation continuously for up to 1 hour.
- **Retry Attempts:** Retry Attempts are configured to 2 retries with jittered exponential backoff.
- **Dead-Letter Queue (DLQ):** Failed event invocations that exhaust all retry attempts are routed to an Amazon SQS Dead-Letter Queue (DLQ): arn:aws:sqs:ap-south-1:<ACCOUNT_ID>:intellis-eventbridge-dlq. An accompanying CloudWatch alarm triggers an immediate P1 alert to the on-call DevOps engineer.

6. Amazon DynamoDB State Management & Control Windows

Amazon DynamoDB provides the ultra-low latency, highly available NoSQL database layer for the Intellis scheduling platform. Operating with single-digit millisecond response times, DynamoDB manages real-time plant SCADA meter telemetry, planned maintenance outage control windows, dynamic curtailment orders, multi-generator asset topologies, and revision idempotency state locks. This section documents the table schemas, primary key structures, access patterns, and query optimizations.

6.1 Complete Inventory of Production DynamoDB Tables

The following table defines the five (5) core DynamoDB tables deployed across production and testing environments:

Table Name	Partition Key (HASH)	Sort Key (RANGE)	Item Count (Typical)	Core Functional Scope
global1-site-telemetry	site_id (String)	event_ts (String)	14,800+ items	Stores high-frequency SCADA meter data: Active MW, Irradiance W/m2, Temperature, Wind Speed.
plant_control_windows1 (Prod) / plant_control_windows_test	plant_id / site_id (String)	window_id (String)	Active Outage Windows	Stores planned maintenance outages, inverter shutdowns, grid curtailments, and capacity derates.
plant_control_state1 (Prod) / plant_control_state_test	plant_id / site_id (String)	None (Single Item/Site)	1 per Site	Maintains real-time plant state, current revision lock, trip status, and active operational mode.

multi_generator_plant	plant_id (String)	None (Single Item/Park)	Registered Multi-Gen Parks	Defines sub-generator asset topologies, nameplate capacities, and PPA column mapping.
weather_model_intelligence	record_id (String)	None	MOS Model Weights	Maintains 7-day historical Model Output Statistics (MOS) scores and diurnal ranking weights.

6.2 Detailed Table Schemas & JSON Data Contracts

To enable junior developers and cloud engineers to interact programmatically with DynamoDB, the exact JSON item schemas are specified below:

SCADA Telemetry Item: 1. Real-Time Telemetry Item Schema (global1-site-telemetry):

```
{
  "site_id": {"S": "ZTRIC"},
  "event_ts": {"S": "2026-10-08T14:15:00+05:30"},
  "active_power_mw": {"N": "28.45"},
  "poa_irradiance": {"N": "685.2"},
  "ambient_temp_c": {"N": "32.4"},
  "wind_speed_ms": {"N": "3.8"},
  "grid_frequency_hz": {"N": "50.02"},
  "source": {"S": "SCADA_METER_PRIMARY"}
}
```

Control Window Item: 2. Planned Outage & Control Window Item Schema (plant_control_windows1):

```
{
  "plant_id": {"S": "JEWLI"},
  "window_id": {"S": "OUTAGE-20261009-WTG07"},
  "start_time": {"S": "2026-10-09T08:00:00+05:30"},
  "end_time": {"S": "2026-10-09T16:00:00+05:30"},
  "outage_type": {"S": "TURBINE_MAINTENANCE"},
  "affected_units": {"SS": ["WTG-07", "WTG-12"]},
  "capacity_derate_mw": {"N": "7.20"},
  "active_status": {"BOOL": true}
}
```

Multi-Generator Topology Item: 3. Multi-Generator Asset Topology Item Schema (multi_generator_plant):

```
{
  "plant_id": {"S": "SHAHA"},
  "total_ac_capacity_mw": {"N": "40.0"},
  "sub_generators": {
    "L": [
      {"M": {"asset_id": {"S": "SIDDEHESH"}, "capacity_mw": {"N": "15.0"}, "column_name": {"S": "SIDDEHESH_MW"}}},
      {"M": {"asset_id": {"S": "PRANAV"}, "capacity_mw": {"N": "15.0"}, "column_name": {"S": "PRANAV_MW"}}},
      {"M": {"asset_id": {"S": "LOKGREENB2"}, "capacity_mw": {"N": "10.0"}, "column_name":
```

```

{"S": "LOKGREENB2_MW"}}}
]
}
}

```

6.3 DynamoDB Query Optimization & Idempotency Locking

High-Performance Query Patterns: To prevent costly full-table scans, all Lambda interactions execute optimized Query requests using composite KeyConditionExpression parameters: KeyConditionExpression = 'site_id = :s AND event_ts BETWEEN :t1 AND :t2'. Responses are returned in under 4 milliseconds.

Revision Idempotency State Locking: To prevent duplicate schedule generation during transient EventBridge retries, Lambda functions execute atomic Conditional Writes to plant_control_state1 using ConditionExpression = 'attribute_not_exists(revision_id)'. If a revision for that 15-minute gate has already been processed, the write fails with ConditionalCheckFailedException, causing the duplicate invocation to terminate safely without overwriting the statutory schedule.

7. Amazon CloudWatch Observability, Logging & Alerting

Amazon CloudWatch delivers comprehensive observability, operational telemetry, structured log analysis, and automated incident alerting across all Intellis cloud services. By centralizing telemetry across 82+ microservices, CloudWatch enables engineering and operations teams to monitor grid schedule generation in real time, detect anomalies, and enforce strict SLAs. This section details the log group hierarchy, structured JSON logging schemas, custom metrics, and alarm matrices.

7.1 Log Group Hierarchy & Retention Governance

Every Lambda microservice streams stdout/stderr directly to a dedicated CloudWatch Log Group within the /aws/lambda/ namespace. The production account manages 82+ dedicated log groups:

Log Group Namespace	Target Functions	Retention Policy	Operational Purpose
/aws/lambda/{SITE}-ai-intellis-scheduler	27+ Intraday site microservices	90 Days Active Retention	Captures 15-minute real-time revision telemetry, SCADA deltas, and S3 upload URIs.
/aws/lambda/intellis-dayhead-forecast-solar	Day-Ahead Solar centralized engine	90 Days Active Retention	Captures daily 96-block clear-sky calculations, 143 NWP model fetches, and MOS rankings.
/aws/lambda/intellis-dayhead-forecast-wind	Day-Ahead Wind centralized engine	90 Days Active Retention	Captures hub-height wind speeds, IEC 61400-12 air densities, and OEM power transformations.
/aws/lambda/intellis-weekhead-forecast-solar	Week-Ahead Solar planning engine	90 Days Active Retention	Captures 672-block ephemeris, 168h NWP splines, and 21-slot diurnal ensembling logs.
/aws/lambda/intellis-weekhead-forecast-wind	Week-Ahead Wind planning engine	90 Days Active Retention	Captures 168h wind splines, 28-regime ensembling, and park derating logs.
/aws/lambda/{SITE}-meter-	SCADA telemetry fetcher	30 Days Active	Captures raw meter poll responses,

DE_SOLAR-fetcher	Lambdas	Retention	parsing status, and DynamoDB put latencies.
------------------	---------	-----------	---

7.2 Structured JSON Logging Standard

Unstructured plain-text logging is strictly prohibited in production code. All Intellis modules utilize Python's json logger to emit structured JSON events. This enables CloudWatch Logs Insights to execute fast, indexed SQL-like queries:

Structured JSON Log Format: Standard Production Telemetry Log Schema:

```
{
  "timestamp": "2026-10-08T14:15:04.281Z",
  "level": "INFO",
  "environment": "PRODUCTION",
  "service": "intellis-scheduler",
  "site_id": "ZTRIC",
  "schedule_type": "INTRADAY",
  "revision_number": "Rev04",
  "execution_duration_ms": 4820,
  "blocks_generated": 96,
  "max_mw": 48.25,
  "min_mw": 0.00,
  "guardrails": {
    "night_zero_count": 42,
    "ceiling_clamps": 3,
    "ramp_adjustments": 0
  },
  "s3_destination": "s3://vedanjay-schedules1/generated/vedanjay_ai_intellis/ZTRIC/2026-10-08/ZTRIC_Rev04_1415.csv",
  "status": "SUCCESS"
}
```

7.3 Operational CloudWatch Alarms & Severity Matrix

Automated CloudWatch Alarms continuously monitor operational health, dispatching immediate SNS notifications to PagerDuty and Slack alerting channels:

Alarm Name	Monitored Metric	Evaluation Threshold	Severity Tier	Automated Action / Escalation
LambdaExecutionErrorAlarm	Errors (Sum)	>= 1 in 5 minutes	P1 CRITICAL	Immediate SMS & PagerDuty escalation; triggers on-call engineer.
MissingScheduleAtGateClosure	ScheduleMissingFlag	>= 1 at Gate - 10 min	P1 CRITICAL	Triggers automated fallback dispatcher to submit last-known valid schedule.
LambdaDurationHighAlarm	Duration (Maximum)	> 600,000 ms (10)	P2 HIGH	CloudWatch notification to DevOps

		min)		channel; investigates API timeouts.
ExternalWeatherAPIFailureAlarm	API_5xx_Errors	>= 3 in 15 minutes	P2 HIGH	Switches weather ingestion layer to local historical climatological cache.
S3PutLatencyHighAlarm	S3UploadDuration	> 2,000 ms	P3 MEDIUM	Logs S3 throttling warning; verifies AWS regional endpoint health.

7.4 Operational CloudWatch Dashboard Specifications

A centralized CloudWatch Dashboard (Intellis-Executive-Operations) aggregates real-time metrics across all sites:

- **Fleet Status Heatmap:** Widget 1: Real-Time Revision Heatmap displaying 15-minute dispatch status across all 27+ sites (Green = Success, Red = Error).
- **Compute Execution Latency:** Widget 2: Line chart tracking Lambda execution duration (p50, p90, p99) against the 15-second statutory SLA.
- **Physics Safety Clamping:** Widget 3: Bar chart displaying the count of physical guardrails triggered (night zeroing, clear-sky caps, ramp rate limits).
- **External Weather Ingestion Health:** Widget 4: External API health monitoring Open-Meteo HTTP 200 latency and rate-limit headroom.

8. Build, Deployment & Zero-Downtime Rollout Procedures

Deploying updates across 29+ production serverless functions while maintaining continuous 15-minute grid dispatch operations requires strict, automated release engineering. This section documents the end-to-end build pipeline, automated multi-Lambda deployment orchestration, zero-downtime rolling update mechanics, and post-deployment validation suites.

8.1 Step-by-Step Container Compilation & Packaging Pipeline

The release engineering lifecycle executes five (5) sequential phases from source code to verified ECR artifact:

- **Phase 1 — Pre-Build Verification:** Execute local unit testing suite verifying astronomical ephemeris equations, Kt cubic spline interpolations, and physical guardrail clamps: `pytest tests/`.
- **Phase 2 — Amazon ECR Registry Authentication:** Authenticate local Docker daemon to the target Amazon ECR registry using AWS CLI credentials: `aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com`.
- **Phase 3 — Cross-Platform Container Compilation:** Compile the container image using Buildx with mandatory flags: `docker buildx build --platform linux/amd64 --provenance=false --load -f Dockerfile.intellis-ai-lambda -t <ECR_IMAGE_URI> .`
- **Phase 4 — Artifact Upload to Amazon ECR:** Push the compiled image to Amazon ECR: `docker push <ECR_IMAGE_URI>`. Verify SHA-256 digest generation in the AWS console or CLI.
- **Phase 5 — Automated Security Vulnerability Scan:** Inspect Amazon Inspector vulnerability scanning report, verifying zero CRITICAL or HIGH CVE findings prior to deployment authorization.

8.2 Automated Multi-Lambda Deployment Orchestration Engine

Rather than updating Lambda functions manually, the production codebase provides an automated deployment orchestration script: `schedule/build_and_deploy_all_lambdas.py`. This script programmatically updates all 29+ serverless microservices:

- **Registry Resolution:** Resolves target AWS account (Testing 608744602858 or Production 397483229292) and constructs exact image URI: `<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/intellis-ai-scheduler:<TAG>`.
- **Sequential Microservice Loop:** Iterates across the master array of 29+ Lambda functions (27 Intraday sites, `intellis-dayhead-forecast-solar`, `intellis-weekhead-forecast-solar`).
- **Atomic Code Update Call:** Calls `lambda_client.update_function_code(FunctionName=fn, ImageUri=IMAGE_URI)` to initiate rolling container replacement.
- **State Polling & Barrier Synchronization:** Polls `lambda_client.get_function(FunctionName=fn)` every 2 seconds, verifying `Configuration.LastUpdateStatus == 'Successful'` before advancing to subsequent functions.
- **Deployment Summary Generation:** Generates a structured deployment summary DataFrame and saves audit trail to `lambda_deployment_summary_<DATE>.csv`.

8.3 Zero-Downtime Deployment Mechanics & Lambda State Polling

A critical advantage of AWS Lambda container deployments is zero-downtime execution. When `update_function_code` is invoked, AWS Lambda does not terminate active instances or disrupt in-flight grid calculations:

- **Phase A — In-Flight Request Protection:** Existing Lambda execution environments continue running the previous container image, processing any active 15-minute grid revision without interruption.
- **Phase B — Background Layer Caching & Warm-Up:** In the background, AWS Lambda pulls the new container image from ECR, unpacks layers into local microVM storage, and runs initial container health checks. The function reports `LastUpdateStatus = 'InProgress'`.
- **Phase C — Atomic Routing Pointer Switch:** Once the new container environments are verified warm, AWS Lambda flips the internal pointer atomically. Subsequent EventBridge triggers route exclusively to the new container image. The function reports `LastUpdateStatus = 'Successful'`.
- **Phase D — Automatic Infrastructure Rollback Guard:** If the container fails initialization (e.g. invalid entrypoint or missing library), AWS Lambda automatically rolls back the pointer to the previous version and reports `LastUpdateStatus = 'Failed'` with `LastUpdateStatusReason`.

8.4 Post-Deployment Verification & Smoke Test Suite

Immediately following multi-Lambda deployment, the release engineer executes the automated smoke testing suite:

- **Test 1 — Direct Synthetic Invocation:** Invoke a test site Lambda (e.g. `ZTRIC-ai-intellis-scheduler`) using a synthetic test payload: `{'site_id': 'ZTRIC', 'bucket': 'vedanjay-schedules-test-608744602858', 'test_mode': true}`.
- **Test 2 — Latency & Memory Verification:** Verify that the Lambda execution returns `StatusCode = 200` within 6.0 seconds without runtime memory warnings.

- **Test 3 — S3 Output Integrity Validation:** Verify that the generated CSV schedule appears in the S3 test bucket with exactly 96 rows, valid timestamps, non-null numbers, and correct headers.
- **Test 4 — Telemetry & Log Inspection:** Query CloudWatch Logs Insights to verify that structured JSON logs report status: 'SUCCESS' and confirm that no unhandled exceptions were logged.

9. Emergency Rollback, Disaster Recovery & Troubleshooting Runbooks

Operating statutory power scheduling infrastructure requires robust contingency engineering. Under CERC regulations, missing a 15-minute grid revision gate incurs severe commercial penalties under Deviation Settlement Mechanism (DSM) accounts. This section provides actionable, step-by-step operational runbooks for immediate production rollbacks, regional disaster recovery, and automated incident troubleshooting.

9.1 Immediate Production Image Rollback Procedure (< 60 Seconds)

If a newly deployed container image introduces numerical divergence, unhandled exceptions, or missing libraries, the on-call engineer executes the immediate multi-Lambda rollback procedure:

- **Step 1: Identify Previous Stable Tag:** Inspect the production deployment registry (`lambda_deployment_summary_*.csv`) or query ECR to identify the previous verified image tag (e.g. `intellis-dayahead-20261003-v3`).
- **Step 2: Execute Emergency Rollback Script:** Run the automated rollback command: `python schedule/rollback_all_lambdas.py --tag <PREVIOUS_STABLE_TAG> --profile vedanjay-prod`.
- **Step 3: Programmatic API Direct Rollback:** Alternatively, roll back a single critical site Lambda via AWS CLI in 5 seconds:
`aws lambda update-function-code --function-name ZTRIC-ai-intellis-scheduler --image-uri 3974832292.dkr.ecr.ap-south-1.amazonaws.com/intellis-ai-scheduler:intellis-ai-20261005-v1 --region ap-south-1`.
- **Step 4: Verify Restoration of Service:** Poll CloudWatch Logs to confirm that the restored image initializes cleanly and emits status: 'SUCCESS' on the subsequent EventBridge trigger.

9.2 EventBridge Emergency Rule Management & Fallback Freezing

During planned grid-wide maintenance, regional grid blackout conditions, or transmission line tripping, grid operators may instruct generators to freeze scheduling revisions. The operations team can disable or re-enable EventBridge rules instantaneously:

- **Disable Specific Site Rule:** `aws events disable-rule --name ZTRIC-ai-intellis-scheduler-cron-15 --region ap-south-1`. Freezes scheduled triggers immediately.
- **Re-Enable Site Rule:** `aws events enable-rule --name ZTRIC-ai-intellis-scheduler-cron-15 --region ap-south-1`. Resumes normal 15-minute dispatch.
- **Statutory Baseline Freezing:** If a site's real-time schedule fails to generate within 10 minutes of gate closure, the fallback daemon inverts to the Day-Ahead baseline schedule for that block, ensuring non-zero grid submission.

9.3 Troubleshooting Runbook Matrix for Critical Cloud Incidents

The following troubleshooting matrix codifies root causes, diagnostic commands, and automated remediations for six (6) critical cloud operational failure scenarios:

Incident Scenario	Root Cause & Symptoms	Diagnostic CLI Command	Immediate Remediation Procedure
Lambda Out-of-Memory (OOM)	Memory utilization exceeds 512 MB during large 143-model matrix ensembling.	aws logs filter-log-events --log-group-name ... --filter-pattern 'Task timed out'	Increase memory allocation to 1024 MB via update-function-configuration; memory updates apply in < 2 seconds.
External NWP API HTTP 429	Exceeded Open-Meteo commercial concurrent API rate quota.	aws logs filter-log-events --log-group-name ... --filter-pattern '429 Too Many Requests'	Engine activates local /tmp/ disk cache; switches API key to secondary failover key stored in AWS Secrets Manager.
DynamoDB Idempotency Collision	Two triggers fired concurrently within same 15-minute block.	aws logs filter-log-events --log-group-name ... --filter-pattern 'ConditionalCheckFailed'	Normal system behavior: secondary duplicate invocation safely exits with code 200 without overwriting state.
S3 Access Denied (403)	IAM execution role policy drifted or missing S3 PutObject permission.	aws logs filter-log-events --log-group-name ... --filter-pattern 'AccessDenied'	Verify global1-lambda-role inline policy; confirm bucket policy does not enforce conflicting Deny statements.
ECR Attestation Rejection	Docker built without --provenance=false flag, causing multi-manifest index.	aws lambda get-function --function-name ... (LastUpdateStatusReason)	Rebuild container with --provenance=false flag and push under incremented tag; re-deploy Lambda function.
System Time Clock Drift	Local developer machine in non-IST timezone generating skewed block indices.	aws logs filter-log-events --log-group-name ... --filter-pattern 'block_of_day'	Ensure environment variable TZ=Asia/Kolkata is injected; force pytz.timezone('Asia/Kolkata') in code.

9.4 Regional Disaster Recovery & Business Continuity Architecture

To ensure business continuity in the catastrophic event of an AWS ap-south-1 (Mumbai) regional outage, the Intellis disaster recovery strategy incorporates secondary regional replication to ap-south-2 (Hyderabad):

- **Cross-Region S3 Replication (CRR):** Critical schedule output prefixes in vedanjay-schedules1 are replicated continuously to a disaster recovery bucket in ap-south-2 via S3 Cross-Region Replication (CRR).
- **DynamoDB Global Table Readiness:** Core telemetry and control tables (global1-site-telemetry, plant_control_windows1) can be converted to DynamoDB Global Tables, maintaining multi-region active-active synchronization with sub-second replication latency.
- **Infrastructure-as-Code (IaC) Recovery:** AWS CloudFormation / Terraform Infrastructure-as-Code (IaC) templates are maintained in the repository, enabling full compute redeployment in ap-south-2 within fifteen (15) minutes.

10. Developer & Intern Infrastructure Onboarding Blueprint

Section 10 provides a comprehensive, hands-on onboarding guide for new cloud engineers, DevOps practitioners, and interns joining the Intellis Renewable Power Scheduling team. By following this sequential blueprint, an engineer can configure their workstation, compile container images, test Lambda execution locally via the AWS Runtime Interface Emulator (RIE), deploy microservices to the testing account, and promote verified builds to production.

10.1 Ten-Step Chronological Onboarding & Deployment Roadmap

The following ten-step sequence guides developers from zero local setup to successful cloud deployment:

- **Step 1: Workstation Prerequisite Installation:** Install Python 3.12 (64-bit), Docker Desktop 4.25+, and AWS CLI v2. Ensure Docker daemon is running with Buildx support enabled.
- **Step 2: AWS CLI Profile Configuration:** Configure local AWS credentials for testing (profile intellis-608 / Test_Vedanjay2) and production (profile vedanjay-prod). Verify STS caller identity via `aws sts get-caller-identity --profile intellis-608`.
- **Step 3: Repository Clone & Structure Navigation:** Clone git repository ai-schedule. Navigate to `schedule/` directory containing `Dockerfile.intellis-ai-lambda` and deployment scripts.
- **Step 4: Local Virtual Environment Setup:** Create dedicated Python 3.12 virtualenv: `python -m venv venv; activate venv; pip install -r requirements.lambda.txt`.
- **Step 5: Local Container Compilation via Buildx:** Build the test container image locally: `docker buildx build --platform linux/amd64 --provenance=false --load -f Dockerfile.intellis-ai-lambda -t intellis-ai-local:latest`.
- **Step 6: Local Execution Testing via Lambda RIE:** Run container locally on port 8080: `docker run -p 8080:8080 intellis-ai-local:latest`. Send synthetic invocation via curl and verify HTTP 200 response.
- **Step 7: Amazon ECR Authentication & Image Push:** Authenticate to testing ECR registry: `aws ecr get-login-password --profile intellis-608 | docker login --username AWS --password-stdin 608744602858.dkr.ecr.ap-south-1.amazonaws.com`. Tag and push image.
- **Step 8: Deploy to Testing Account Lambdas:** Run `deploy_dayahead_lambda.py` or `build_and_deploy_all_lambdas.py` targeting the testing account to update Lambda code ImageUris.
- **Step 9: End-to-End Schedule Validation:** Trigger a test EventBridge execution or direct Lambda invoke; verify that the resulting 96-block or 672-block CSV is correctly written to `s3://vedanjay-schedules-test-608744602858/`.
- **Step 10: Production Promotion & Change Authorization:** Following successful testing validation, submit deployment pull request with Change Request Ticket; promote image to production registry and execute rolling deployment under supervision.

10.2 AWS CLI Profile Configuration Standard

Engineers must configure their local `~/.aws/config` and `~/.aws/credentials` files following the standardized profile taxonomy:

AWS Config File: Standard `~/.aws/config` Configuration:

```
[profile intellis-608]
region = ap-south-1
output = json
```

```
[profile vedanjay-prod]
region = ap-south-1
output = json
```

AWS Credentials File: Standard ~/.aws/credentials Configuration:

```
[intellis-608]
aws_access_key_id = AKIAXXXXXXXXXXXXXXXXXX
aws_secret_access_key = XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

[vedanjay-prod]
aws_access_key_id = AKIAYYYYYYYYYYYYYYYYYY
aws_secret_access_key = YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY
```

10.3 Local Testing via AWS Lambda Runtime Interface Emulator (RIE)

The AWS managed base image includes the Lambda Runtime Interface Emulator (RIE), allowing developers to test function endpoints locally before pushing to ECR:

- **1. Launch Local Container:** `docker run -p 8080:8080 -e SCHEDULE_BUCKET=vedanjay-schedules-test-608744602858 -e TZ=Asia/Kolkata intellis-ai-local:latest`
- **2. Send Synthetic Invocation Request:** `curl -XPOST "http://localhost:8080/2015-03-31/functions/function/invocations" -d '{"site_id":"ZTRIC", "bucket":"vedanjay-schedules-test-608744602858", "output_prefix":"generated/vedanjay_ai_intellis"}'`
- **3. Validate Terminal Telemetry:** Inspect container terminal output: verify that NREL SPA ephemeris completes in < 50ms, NWP splines evaluate without NaN errors, and CSV output reports status 200.

10.4 Architectural Certification, Regulatory Compliance & Engineering Sign-Off

This Cloud Migration Infrastructure and Deployment Guide constitutes the official cloud engineering architecture for the Intellis Renewable Energy Scheduling System. The container topologies, multi-account boundaries, EventBridge cron rules, and zero-downtime deployment pipelines specified herein satisfy all statutory requirements of the Indian Electricity Grid Code (IEGC 2023) and ISO/IEC 27001 cloud security standards.